OOP Concepts

Formal Definitions + C++ Examples

A clean interview reference covering the core OOP vocabulary, the four pillars, polymorphism, object lifecycle, copying, relationships, static members, and inheritance types.

Use this for: quick revision before a face-to-face C++ interview. The definitions are phrased so they can be spoken directly.

1. OOP Foundations

1. Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming paradigm that organizes software around objects that encapsulate state and behavior, and uses mechanisms such as encapsulation, abstraction, inheritance, and polymorphism to model and manage complex systems.

Example

```
class Student
{
private:
    int rollNo;

public:
    void display()
    {
        cout << rollNo;
    }
};
```

2. Class

A class is a user-defined data type that defines the structure and behavior of objects through data members and member functions.

Example

```
class Student
{
    int rollNo;

public:
    void display();
};
```

Interview point: A class is a blueprint; an object is an instance created from that blueprint.

3. Object

An object is a concrete instance of a class that has its own state and can invoke the behavior defined by the class.

Example

```
Student s1;
Student s2;
```

Interview point: Each object has its own instance data.

2. Four Pillars of OOP

4. Encapsulation

Encapsulation is the bundling of data and the operations that manipulate that data within a single unit, typically a class, while controlling direct access to the object's internal state.

Example

```
class Bank
{
private:
    double balance;

public:
    void deposit(double amount)
    {
        if (amount > 0)
            balance += amount;
    }
};
```

Interview point: Think: bundle data + behavior, then provide controlled access.

5. Abstraction

Abstraction is the process of exposing the essential interface or behavior of an object while hiding unnecessary implementation details.

Example

```
class Payment
{
public:
    virtual void pay() = 0;
};
```

Interview point: Think: expose what an object does; hide how it does it.

6. Inheritance

Inheritance is a mechanism in which a derived class acquires accessible members and behavior from a base class and can extend or specialize that behavior.

Example

```
class Person
{
};

class Student : public Person
{
};
```

Interview point: Inheritance should model an is-a relationship: Student IS-A Person.

7. Polymorphism

Polymorphism is the ability to use a common interface or operation while allowing different object types or contexts to provide different behavior.

Example

```
Person* p = new Student();
p->introduce();
```

Interview point: C++ commonly distinguishes compile-time and run-time polymorphism.

3. Polymorphism and Dynamic Dispatch

8. Compile-Time Polymorphism

Compile-time polymorphism is polymorphism in which the compiler determines the applicable operation during compilation.

Example

```
int add(int a, int b);
int add(int a, int b, int c);

Complex c3 = c1 + c2;
```

Interview point: Function overloading and operator overloading are common examples.

9. Function Overloading

Function overloading is defining multiple functions with the same name in the same scope but with different parameter lists.

Example

```
int add(int a, int b);
int add(int a, int b, int c);
double add(double a, double b);
```

Interview point: Changing only the return type does not constitute overloading.

10. Operator Overloading

Operator overloading is defining the behavior of an existing operator for a user-defined type.

Example

```
class Complex
{
public:
    Complex operator+(const Complex& other)
    {
        return Complex(real + other.real,
                       imag + other.imag);
    }
};
```

Interview point: This lets expressions such as c1 + c2 work for Complex objects.

11. Run-Time Polymorphism

Run-time polymorphism is polymorphism in which a virtual function call through a base-class pointer or reference is dynamically dispatched according to the object's actual derived type.

Example

```
class Person
{
public:
    virtual void introduce()
    {
        cout << "Person";
    }
};

class Student : public Person
{
public:
    void introduce() override
    {
        cout << "Student";
    }
};

Person* p = new Student();
p->introduce();
```

Interview point: The base pointer refers to a Student, so Student::introduce() executes.

12. Function Overriding

Function overriding occurs when a derived class provides its own implementation of a virtual base-class function with a compatible signature.

Example

```
class Person
{
public:
    virtual void introduce();
};

class Student : public Person
{
public:
    void introduce() override;
};
```

Interview point: Overloading changes the parameter list; overriding replaces inherited virtual behavior.

13. Virtual Function

A virtual function is a base-class member function that enables dynamic dispatch when called through a base-class pointer or reference.

Example

```
class Animal
{
public:
    virtual void sound()
    {
        cout << "Animal";
    }
};
```

14. Pure Virtual Function

A pure virtual function is a virtual function declared with = 0 that makes the containing class abstract and establishes a required interface for derived classes.

Example

```
class Shape
{
public:
    virtual void area() = 0;
};
```

15. Abstract Class

An abstract class is a class that cannot be instantiated directly and is intended to provide a common interface or partial implementation for derived classes. In C++, a class containing at least one pure virtual function is abstract.

Example

```
class Shape
{
public:
    virtual void area() = 0;
};

// Shape s;                 // ERROR
Shape* p = new Circle();    // OK if Circle implements area
```

4. Object Lifecycle and Copying

16. Constructor

A constructor is a special member function that is automatically invoked when an object is initialized and is used to establish its initial state.

Example

```
class Student
{
public:
    Student(int rollNo)
    {
        // initialize object
    }
};
```

Interview point: A constructor has the class name and no return type.

17. Destructor

A destructor is a special member function that is automatically invoked when an object is destroyed and is used to release resources or perform cleanup associated with the object.

Example

```
class Student
{
public:
```

```
    ~Student()
    {
        // cleanup
    }
};
```

Interview point: A destructor takes no parameters. A polymorphic base class should generally have a virtual destructor.

18. Constructor Overloading

Constructor overloading is defining multiple constructors in a class with different parameter lists, allowing objects to be initialized in different ways.

Example

```
Student();
Student(int rollNo);
Student(int rollNo, char grade);
```

19. Copy Constructor

A copy constructor is a constructor that initializes a new object from an existing object of the same class, typically taking the source object by const reference.

Example

```
Student(const Student& other)
{
    // copy members from other
}

Student s1;
Student s2 = s1;
```

20. Shallow Copy

A shallow copy copies member values as they are, including pointer values, so copied objects may refer to the same dynamically allocated resource.

Example

```
Student(const Student& other)
{
    marks = other.marks;   // shallow copy
}
```

Interview point: Both objects can point to the same memory; this can cause shared state and double deletion when ownership is involved.

21. Deep Copy

A deep copy creates independent copies of dynamically allocated resources so that the destination object owns separate resources from the source.

Example

```
Student(const Student& other)
{
    marks = new int(*other.marks);  // deep copy
}
```

Interview point: The pointers are different, even though the pointed-to values initially match.

22. this Pointer

The this pointer is an implicit pointer available in non-static member functions that refers to the current object.

Example

```
class Student
{
    int age;

public:
    void setAge(int age)
```

```
    {
        this->age = age;
    }
};
```

Interview point: this->age refers to the member; age refers to the parameter.

5. Access Control and Object Relationships

23. Access Specifiers

Access specifiers determine where class members can be accessed. C++ provides public, protected, and private access control.

Example

```
class Example
{
private:
    int a;

protected:
    int b;

public:
    int c;
};
```

Interview point: private: class/friends; protected: class/derived classes/friends; public: accessible through the public interface.

24. Association

Association is a relationship in which objects of two classes interact or use one another without implying ownership or a shared lifetime.

Example

```
teacher.updateStudentGrade(student);
```

Interview point: The teacher uses the student object, but neither object necessarily owns the other.

25. Aggregation

Aggregation is a weak has-a relationship in which a container references or groups objects whose lifetimes are independent of the container.

Example

```
class Department
{
    vector<Teacher*> teachers;
};
```

Interview point: Teachers can continue to exist even if the Department object is destroyed.

26. Composition

Composition is a strong has-a relationship in which the containing object owns a constituent object and the constituent's lifetime is tied to the containing object's lifetime.

Example

```
class House
{
private:
    Room room;
};
```

Interview point: Think: strong ownership and dependent lifetime.

6. Static Members

27. Static Data Member

A static data member is a class-level member for which a single shared instance exists for the class rather than one separate instance per object.

Example

```
class Student
{
    static int count;
};

int Student::count = 0;
```

Interview point: Useful for object counters and other class-wide state.

28. Static Member Function

A static member function belongs to the class rather than a particular object and can be called without an object. It has no this pointer and cannot directly access non-static members.

Example

```
class Student
{
public:
    static int getCount();
};

Student::getCount();
```

7. Types of Inheritance

29. Single Inheritance

Single inheritance occurs when one derived class inherits directly from one base class.

Example

```
Person
   |
 Student
```

30. Multiple Inheritance

Multiple inheritance occurs when one derived class inherits from more than one base class.

Example

```
Person     Bank
   \       /
    Teacher
```

31. Multilevel Inheritance

Multilevel inheritance occurs when a derived class becomes the base for another class, forming multiple levels of inheritance.

Example

```
Person
   |
Student
   |
GraduateStudent
```

32. Hierarchical Inheritance

Hierarchical inheritance occurs when multiple derived classes inherit from the same base class.

Example

```
Person
       /    \
```

```
Student   Teacher
```

33. Hybrid Inheritance

Hybrid inheritance is a combination of two or more inheritance patterns within the same inheritance hierarchy.

Example

```
Example:
multiple + multilevel inheritance
```

8. Interview Cheat Sheet

Four Pillars

Concept	Interview definition
Encapsulation	Bundling data and related methods together while controlling access to internal state.
Abstraction	Exposing essential behavior while hiding unnecessary implementation details.
Inheritance	Deriving a class from an existing class to reuse and specialize behavior.
Polymorphism	Using a common interface while allowing different implementations or behaviors.

Polymorphism

```
Polymorphism
■■■ Compile-time
■    ■■■ Function overloading
■    ■■■ Operator overloading
■
■■■ Run-time
    ■■■ Function overriding + virtual functions
```

High-value distinctions: overloading vs overriding; abstraction vs encapsulation; shallow vs deep copy; aggregation vs composition; static vs non-static members; and base-pointer runtime polymorphism.